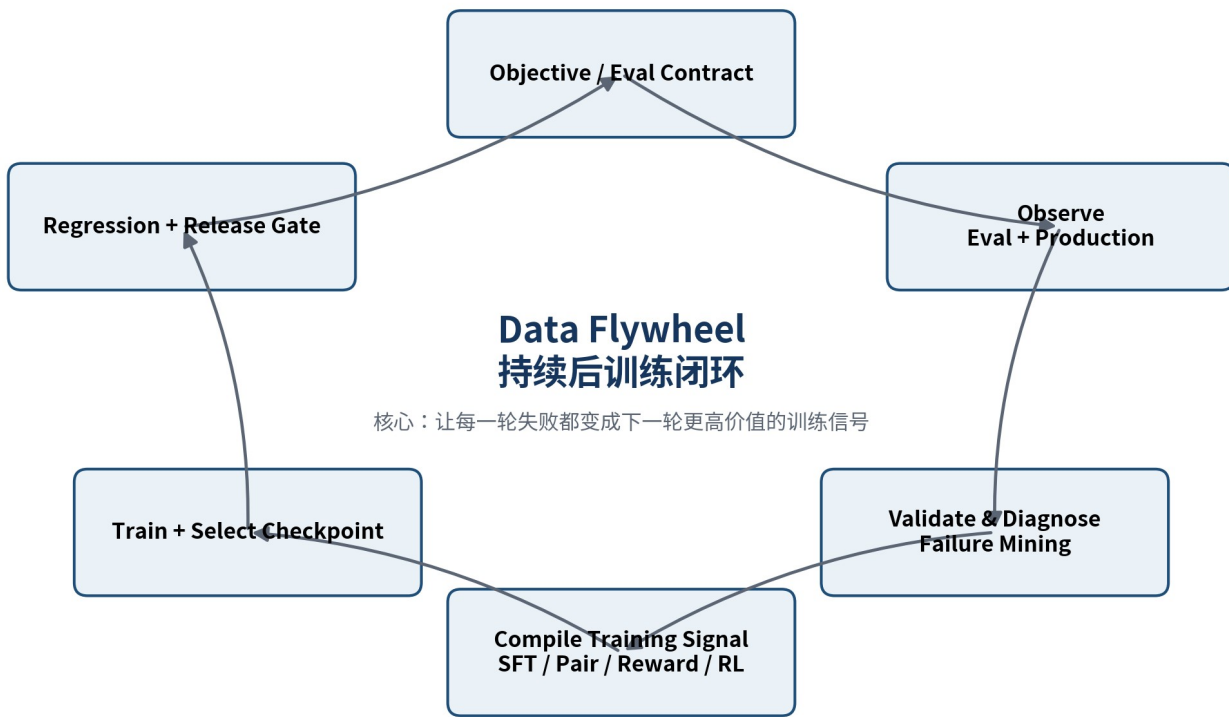


LLM 后训练之 Data Flywheel

数据飞轮与持续后训练闭环

深度研究与工程手册



核心问题：如何让模型的每一轮真实失败，转化为下一轮更高价值的训练信号？

研究范围：Eval • Failure Mining • Data Compiler • SFT/DPO/RLVR • Regression • Production Feedback

0. 执行摘要：Data Flywheel 不是 “不断收集更多数据”

第一性定义： Data Flywheel 是一个闭环控制系统：持续观察当前模型与系统的真实失败，验证失败是否可信，定位根因，将其编译为与训练阶段匹配的数据/奖励/轨迹，再通过回归与生产信号确认修复是否真实且没有破坏旧能力。

Objective → Observe → Validate → Diagnose → Prioritize → Compile → Train → Regression → Deploy → Observe

OpenAI 在 2025 年的 evals 指南中明确提出 “Specify → Measure → Improve”，并把 data flywheel 描述为：记录输入、输出与结果；定期抽样日志；把模糊或高成本案例路由给专家；将专家判断加入 eval 与 error analysis；再更新 prompt、tool 或 model。[1] Anthropic 对 Agent Eval 的实践则强调 capability eval 与 regression eval 的分工：前者提供 “山坡”，后者保护已经获得的能力。[2]

层	核心问题	产物	失败时的典型后果
Observe	哪里变差/哪里没学会？	Eval、生产日志、用户反馈、红队	只看 benchmark，错过真实分布
Validate	这真的是模型失败吗？	Validated Failure	把 grader/harness bug 回灌进训练
Diagnose	为什么失败？	Failure Taxonomy / Root Cause	用错误训练阶段修问题
Prioritize	先修哪个？	Gap Queue	被高频但低价值问题淹没
Compile	如何变成训练信号？	SFT/Pair/Reward/Trajectory	“bad case dump” 不能直接训练
Train	哪种优化最匹配？	Checkpoint / Candidate	SFT 解决不了探索，RL 解决不了规格错误
Regression	修好后破坏了什么？	Release Gate / Replay Bank	局部提升、全局退化
Deploy	真实用户是否受益？	Product Metric / New Failures	离线变好，产品变差

整本手册的核心判断： Data Flywheel 的竞争力不是 “闭环图画得多漂亮”，而是四件事能否同时成立：失败信号可信、训练路由正确、数据保持 near-policy / fresh、旧能力有独立 regression/replay anchor。

1. 为什么 Data Pipeline 不等于 Data Flywheel

Data Pipeline	Data Flywheel
输入数据是预先定义的	输入数据由当前模型的失败动态决定
目标是稳定生产数据	目标是最大化模型的边际学习收益
数据流通常单向	模型行为会反过来改变下一轮数据分布
质量控制主要是 clean/filter	还包括 root-cause、policy relevance、learning frontier
版本更新是人为批次	可以是 eval/production 信号触发的持续更新

Data Pipeline	Data Flywheel
主要 KPI: 吞吐、样本量	主要 KPI: 有效增益 / 总成本 / 回归风险

因此，真正的飞轮不是“新数据 → 新模型”这么简单，而是一个带状态的控制器。当前 policy θ_t 决定它会在哪些任务上失败；这些失败决定下一批训练数据 D_{t+1} ； D_{t+1} 再决定 θ_{t+1} 。训练分布本身是模型状态的函数。

$$D_{(t+1)} = \text{Controller}(\text{Eval}(\theta_t), \text{Production}(\theta_t), \text{Failures}, \text{Cost}, \text{Risk}, \text{Retention})$$

$$\theta_{(t+1)} = \text{Train}(\theta_t, D_{(t+1)}, \text{Objective}_{(t+1)})$$

这也是为什么静态“万能后训练集”很难长期最优：模型变强后，旧数据会从 learnable frontier 退化成已掌握样本；模型进入新产品场景后，新的长尾与交互失败会出现。

2. Data Flywheel 的七层参考架构

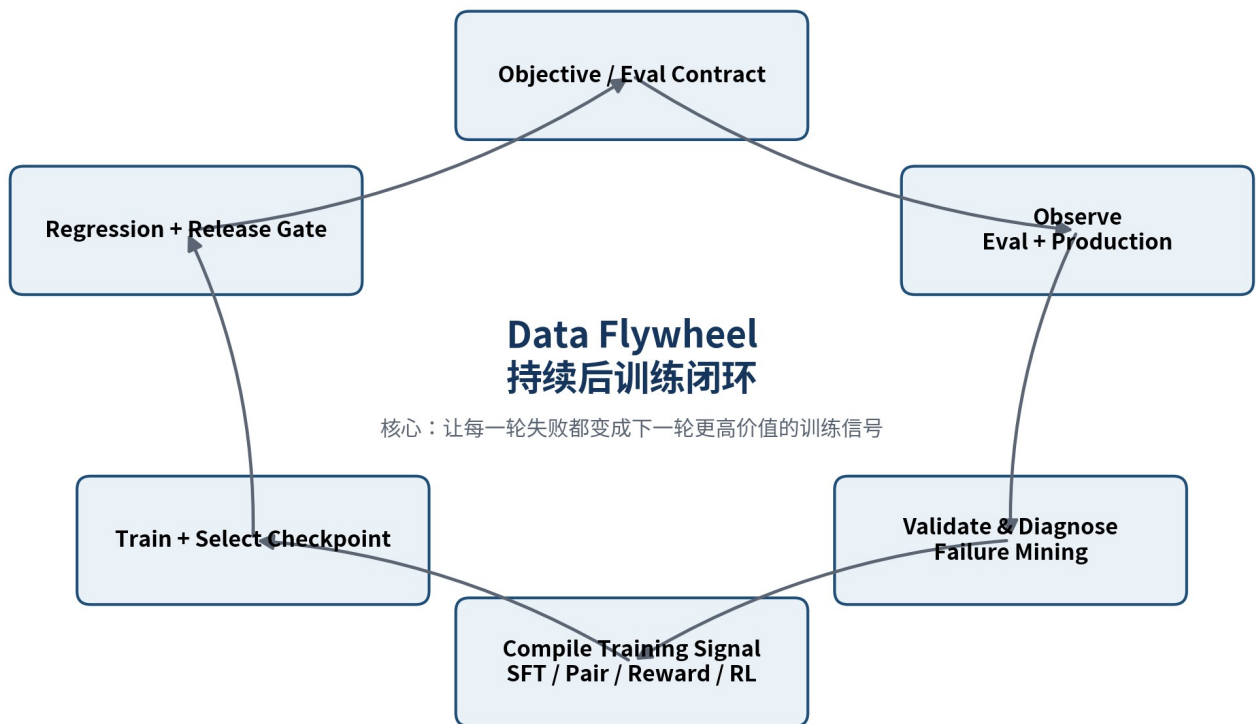


图 1：从 Objective/Eval Contract 到生产反馈的闭环

层	组件	Know-Why	Know-How
L0	Objective / Eval Contract	闭环必须围绕稳定目标，而不是围绕某个 benchmark	维护 capability/behavior/risk spec 与 release thresholds
L1	Signal Collection	真实失败分散在 eval、生产、红队、用户反馈中	统一事件 schema、采样策略和 provenance
L2	Validation & Attribution	Observed failure 不等于 model failure	复现、grader audit、environment audit、root-cause
L3	Gap Queue	预算有限，必须排序	Impact×Frequency×Confidence×Trainability×Novelty / Cost
L4	Data Compiler	不同 failure 需要不同监督信号	SFT / Pair / Reward / Verifier / Trajectory / Distill
L5	Training & Selection	不是所有问题都值得重新训练	离线小实验 →ablation→checkpoint selection
L6	Regression & Release	局部优化可能破坏其他能力	capability + regression + product gates + rollback

OpenAI 对 agent post-training 角色的公开描述也把 RL、数据管线、graders、reward signals、evals、diagnostics 与 model-behavior analysis 放在同一端到端栈中，并强调“构建暴露下一组模型失败的 eval/environment，再把失败转为训练数据、产品修复或新研究方向”。这与飞轮架构高度一致，但它也说明：闭环的执行动作不一定总是“训练模型”，可能是修工具、修 harness、改产品约束。

3. 信号采集：Flywheel 的 Sensor Layer

信号源	优势	偏差/缺点	最适合
Offline capability eval	可控、可复现、便于迭代	可能脱离真实分布	能力爬坡、研究实验
Regression suite	稳定保护旧能力	对新能力信号弱	CI / release gate
Production logs	真实分布、可发现未知失败	无天然 ground truth、隐私/噪声	产品闭环
User feedback	高语义价值、直接反映痛点	稀疏、自选择偏差	高影响 failure
Outcome / business metric	最接近真实价值	因果归因困难	最终验收
Red-team / adversarial agent	主动搜索脆弱区	分布可能过于敌对	安全与鲁棒性
Judge / verifier disagreement	便于发现边界案例	依赖 evaluator 质量	主动学习 / expert routing
Tool/environment telemetry	可定位 agent 失败步骤	需要稳定环境和细粒度日志	Agent / coding / computer use

Know-How：采集层不要直接产出“训练样本”，而先产出 Observation Event。训练资格必须经过后续 Validation 与 Root-Cause；否则 production noise 会直接污染模型。

4. 统一 Failure Event / Data Lineage Contract

如果没有统一事件与 lineage，飞轮很快会变成“谁手里有一份 badcase.csv 谁就训练”。建议每个失败都拥有可追踪的身份与因果链。

字段组	建议字段
Identity	event_id, task_id, user_segment, model_version, harness_version
Input State	prompt/context, tool state, environment snapshot, policy/system prompt
Trajectory	assistant turns, tool calls, observations, intermediate state
Outcome	final answer, environment end state, unit tests, business outcome
Grading	grader_version, verifier result, judge rubric, confidence, disagreement
Failure	taxonomy labels, first_divergence, severity, reproducibility
Provenance	source, collection time, privacy class, consent/retention policy
Lifecycle	queue status, data compiler route, dataset version, train run, regression result

这份 contract 的价值不是“数据工程整齐”，而是允许回答：某个模型提升来自哪类失败？某条训练数据源自哪次生产事件？某个 regression 是否与旧版本 grader 变化有关？一个样本是否在 SFT、DPO 与 Eval 中发生泄漏？

5. Failure Validation：闭环最重要的防污染闸门

关键公式： Observed Failure = Model Error ∪ Eval Error ∪ Harness Error ∪ Environment Noise ∪ Specification Error。只有第一类（或其中一部分）应该进入模型训练。

检查项	问题	处理
Reproducibility	相同 seed / clean env 下能否复现？	不能复现：标记 flaky，不直接训练
Grader Audit	grader 是否拒绝了有效解？	修 grader，重跑历史结果
Harness Audit	tool/schema/timeout/资源限制是否制造失败？	修 harness 或资源契约
Spec Audit	任务要求是否自相矛盾/含糊？	修 task spec，避免“训练模型迎合错误题目”
Counterfactual	换强模型/人工执行是否能完成？	确认可解性与失败归因
Transcript Review	第一处偏离在哪里？	确定是 planning/tool/reasoning/final-answer 哪层

Anthropic 在 2026 Agent Eval 指南中特别强调阅读 transcript，因为低分可能来自 grader bug、agent harness constraint 或任务歧义，而不是模型本身；能力与回归 eval 也必须分离。[2] 这意味着 Failure Mining 与 Data Flywheel 之间必须存在“可信失败”接口，而不能将所有低分自动变成训练数据。

6. Gap Queue：从“失败仓库”到优化优先级

Gap Queue：不要按“失败数量”排序，而按可修复价值排序



Priority ≈ (Impact × Frequency × Confidence × Trainability × Novelty × VerifierReliability) / Cost

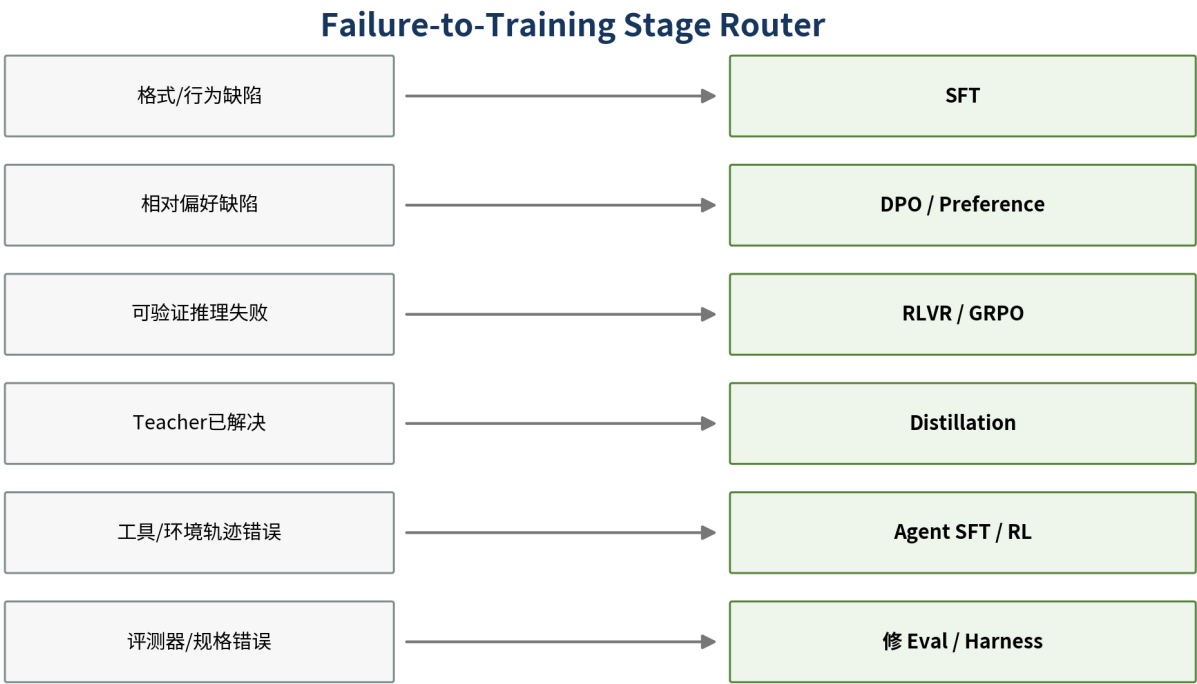
图 2：Gap Queue 的多因子优先级，而非简单按失败次数排序

Priority(f) ≈ Impact × Frequency × Confidence × Trainability × Novelty × VerifierReliability / TotalCost

因子	高值意味着	注意点
Impact	用户/安全/业务损失大	低频高危不能被 frequency 淹没
Frequency	问题覆盖大量真实流量	防止重复同类样本过度采样
Confidence	确实是模型问题且 grader 可信	grader disagreement 可触发专家审阅
Trainability	当前阶段有可用监督信号	不可学的规格/工具问题应路由出去
Novelty	代表新 failure family	避免重复修已知问题
Verifier Reliability	可明确判断修复是否成功	高可靠 verifier 更适合 RLVR/自动闭环
Cost	生成、验证、训练、回归总成本	不要只算训练 FLOPs

2026 的 Learning-Zone Energy (LZE) 把类似思想推进到 RL 在线数据选择：不再均匀对所有 prompt 分配 rollout/gradient 预算，而是关注初始难度、结果不确定性与 pass-rate momentum，使计算集中到 active learning frontier。其论文报告在实验中每步仅保留 40% 训练数据仍可匹配或超过 full-data baseline，并估计降低约 36% 训练 FLOPs。[9]

7. Failure-to-Training Data Compiler



原则：先做 root-cause attribution，再决定“是否训练、用什么训练”

图 3：先做 root-cause，再把 failure 路由到适合的训练阶段

Failure 类型	最自然的训练信号	为什么
不知道“标准行为长什么样”	SFT demonstration	直接建立 behavior prior
两个回答都可行但偏好不同	Preference Pair / DPO	学习相对排序而非唯一答案
数学/代码有客观成败	RLVR / GRPO	允许在线探索与 verifier feedback
Teacher 已有稳定高质量策略	Distillation	把昂贵搜索/大模型行为压入 student

Failure 类型	最自然的训练信号	为什么
Agent 在某步 tool/plan 偏离	Trajectory SFT / Process reward / Agent RL	训练单位是 trajectory，不只是 final answer
Eval/Tool/Spec 本身错误	不训练模型	先修系统，否则闭环会放大错误

Know-Why: “一个 failure 对应一个训练样本”通常是错误抽象。真正需要编译的是 failure family：先生成覆盖同一根因的正例、负例、benign counterpart、hard cases 与 OOD slices，再决定 SFT / pair / reward / trajectory 形态。

8. Data Flywheel 与 Task Distribution：动态混合而不是固定配方

持续训练的 task mixture 应该是 feedback controller 的输出，而不是永久写死的比例。一个实用近似是：

$$w_i(t+1) \propto \text{BasePrior}_i \times \text{Gap}_i(t) \times \text{LearningValue}_i(t) \times \text{Risk}_i \times \text{Reliability}_i / \text{Cost}_i$$

状态	建议权重动作	原因
已掌握且稳定	降采样，但保留 replay / regression	释放预算，同时防遗忘
pass rate 约 30%-70%	显著上采样	通常处于 learnable frontier
几乎全错	先降低难度/加入 curriculum 或 teacher data	直接训练可能无有效梯度
几乎全对	转入 regression bank	继续训练边际价值低
高影响新 failure family	短期 priority boost	快速收敛到最低可接受风险
高不确定 grader	先 expert review / evaluator improvement	避免错误信号被放大

Llama 3 的公开后训练过程提供了“迭代配方”的经典例子：其论文描述了六轮后训练；每轮收集新的 preference/SFT 数据，并从最新模型采样 synthetic data；rejection sampling 通常对每个 prompt 采样 10-30 个输出，由 reward model 选最佳候选。随着模型变强，prompt 复杂度也随之增加，以继续覆盖模型落后的能力。
[4] 这就是手工/半自动 data flywheel 的一种实现。

9. Freshness: 为什么 near-policy data 是飞轮的燃料

持续后训练中的 Freshness - Stability 权衡

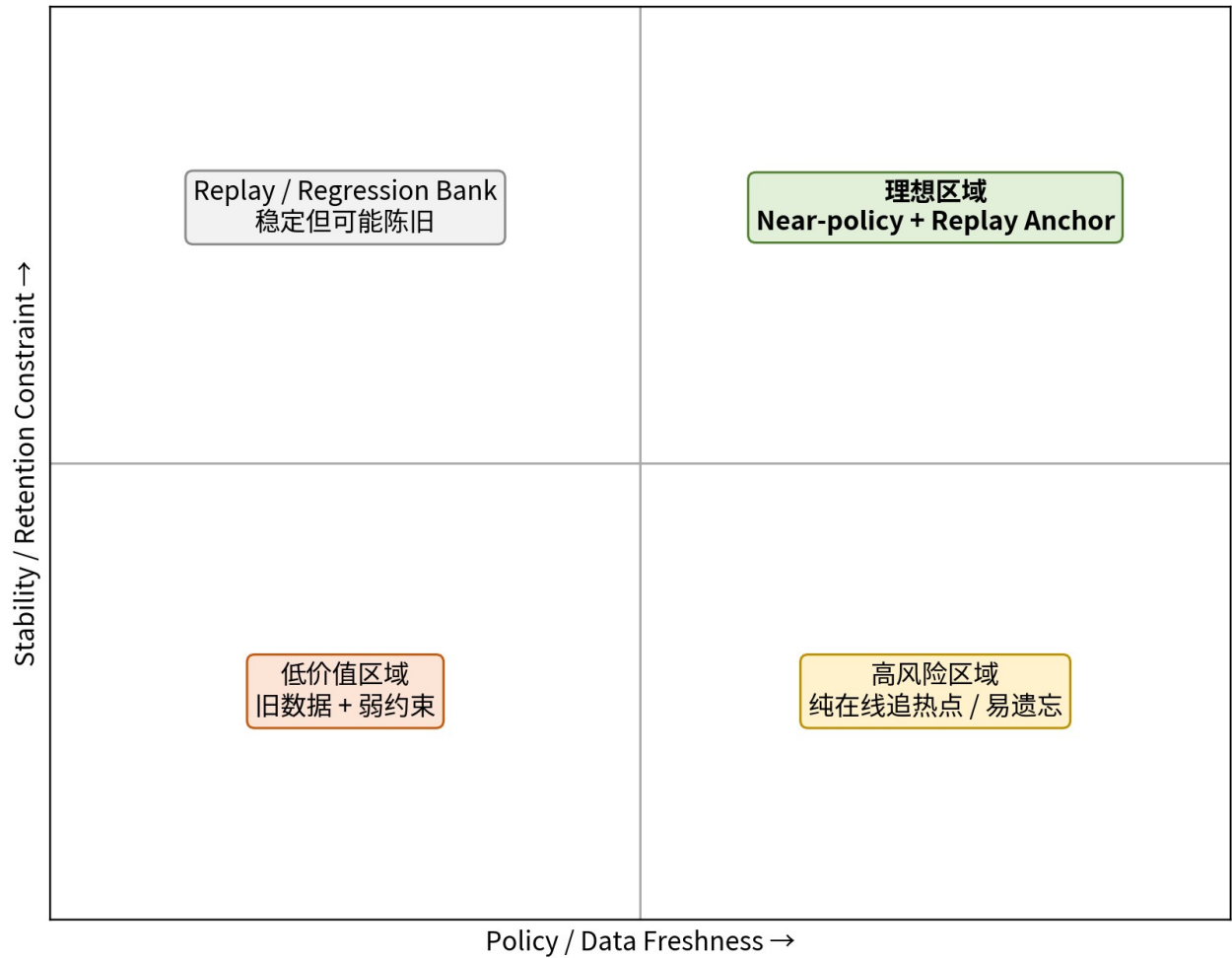


图 4：持续后训练需要同时优化数据新鲜度与能力保持

Preference/RL 数据最容易陈旧：模型 θ_t 产生的数据分布与 θ_{t+k} 的行为分布可能已经不同。旧 pair 仍然能提供原则性偏好，但对当前 policy 的局部梯度信息越来越弱。

数据类型	Freshness 要求	典型用法
General SFT anchor	低-中	长期保持基础行为与风格
Regression replay	中	保护旧能力；需要定期验证仍可代表生产
Preference pair	中-高	最好包含 current/near-policy candidates
RL rollout	最高	通常由当前 policy 生成，控制 staleness
Failure-driven synthetic	高	围绕最新 gap queue 动态生成
Production logs	天然新鲜	需要 validation、privacy 与 label pipeline

Online Preference Alignment 的研究动机也来自 fixed dataset coverage 不足；迭代收集 current policy 的 prompt-response pairs 能扩大探索空间。类似地，ARIES 等 iterative preference 方法把“训练→self-refinement data collection→再训练”做成循环。这里最重要的不是具体算法，而是 data collection policy 变成训练算法的一部分。

10. Retention：持续后训练为什么一定会遇到遗忘

飞轮如果只追新问题，会迅速变成“追热点系统”：每次修一个新 gap，都可能在旧能力上退化。A3 的 2026 自动安全微调系统把这一点做成显式目标：针对安全 failure 自适应生成/加权数据，同时混入大量标准 post-training data，并把 safety failure rate、false positive rate 与 capability benchmark 同时作为优化约束。[5]

防遗忘机制	适用场景	代价/风险
Static replay	通用	简单但可能与当前 policy 风格不匹配
On-policy replay	持续 SFT	更贴近当前模型，但需要 rollout 与 reward/filter
Anchor SFT mix	专项修复	可保护通用行为，但占用训练预算
KL / reference constraint	Preference/RL	稳定 policy，但过强会抑制学习
Regression gates	所有阶段	不直接防遗忘，但能阻止坏 checkpoint 发布
Separate adapter / modular tuning	专项领域	隔离性好，但系统复杂度增加

2026 On-Policy Replay（OPR）进一步把 retention 与 data source 联系起来：用最新 checkpoint 在历史 prompt 上 rollout，按 task reward 过滤，再把保留下来的当前模型响应作为普通 SFT replay。论文强调，其收益来自 on-policy distribution，而不只是“挑高分答案”。[10]

11. Regression Bank：飞轮的“长期记忆”

Eval Bank	目标 pass rate	生命周期	用途
Capability / Frontier	低到中	动态加入难题	告诉团队下一座山在哪里
Regression	接近 100%	长期稳定，版本化	防止已获得能力回退
Safety / High-severity	高且保守	长期固定 + 新威胁增量	release gate
Product contextual	根据业务阈值	随产品/用户变化维护	验证真实效用
OOD / Challenge	较低	定期刷新	检验 generalization
Shadow production	无固定答案	持续抽样	发现分布漂移与未知失败

Anthropic 建议 capability eval 与 regression eval 分开：前者应该让模型“有山可爬”，后者应该几乎全通过；随着 capability eval 被攻克，任务可以“毕业”进入 regression suite。[2] 这实际上提供了 Data Flywheel 的 eval lifecycle：困难任务 → 学习 → 稳定掌握 → 回归保护 → 新困难任务。

Frontier Task → Training Signal → Mastered → Regression Bank → Drift Detection

12. Eval Saturation 与 Frontier Renewal

一个 eval 达到 100% 后，对“继续进步”几乎没有梯度。Flywheel 必须自动发现新的 frontier：

- 1. 对已饱和能力做 difficulty escalation：更长上下文、更复杂约束、更真实环境、更强对手。
- 2. 从生产日志中聚类“高成本/高不确定/低满意度”切片。
- 3. 用 adversarial generator / red-team agent 主动寻找新失效模式。
- 4. 把已有 failure family 做 counterfactual 与 OOD 扩张，而不是只重复原始 badcase。
- 5. 定期审计 grader / verifier，避免模型只是学会 exploit evaluator。

GPT-Red 是一个非常清晰的自改进例子：攻击模型与 defender population 进行 self-play；随着 defenders 变强，red-teamer 被迫发现更强、更丰富的攻击；随后这些攻击被用于生产模型的鲁棒性训练。OpenAI 报告其最新模型在最难直接 prompt-injection benchmark 上相较四个月前的最佳生产模型有 6 倍更少失败。[6] 这体现了“对手变强 → 数据变难 → defender 变强 → 对手继续升级”的 adversarial flywheel。

13. Automated Alignment / A3：从一个失败启动整个微调飞轮

A3 阶段	核心动作	Flywheel 对应模块
Input Failure	单个不良行为例子	Failure Seed
Hypothesis Expansion	生成结构/主题变体假设	Failure Family Mining
Harmful + Benign Generation	同时生成攻击和 benign counterpart	Balanced Data Factory
Train/Val/OOD Split	按 hypothesis 级切分	Leakage-safe Eval Contract
Adaptive Weighting	对当前表现差的 hypothesis 上采样	Dynamic Task Distribution
Post-training Mix	混入标准后训练数据	Retention / Anti-forgetting
Experiment Log	记录每轮 hypothesis、权重、超参、结果	Stateful Controller

A3 的核心并不是某个新的 loss，而是一个 stateful experiment agent：它根据前几轮生成和微调结果调整下一轮 hypothesis、数据权重与超参。其目标同时约束 safety failure rate、false-positive refusal 与 capability retention。[5] 这非常接近“持续后训练控制器”的可执行原型。

14. Tulu 3 / Llama 3：两种典型后训练 Flywheel 形态

维度	Llama 3	Tulu 3
形态	多轮迭代：RS→SFT→DPO，多轮收集新数据	阶段化 pipeline：SFT→DPO→RLVR
数据更新	最新模型生成 synthetic / RS candidates	开放数据混合 + preference + verifiable tasks
Evaluation	能力专项 + 人类偏好/数据质量	development + unseen multi-task eval, decontamination
核心启示	模型变强后，数据复杂度与 candidate generation 一起升级	不同阶段承担不同优化职责，eval 贯穿 recipe
Flywheel 风格	iterative model-data co-evolution	stage-routed post-training architecture

Tulu 3 的贡献之一是把现代 post-training 的数据、SFT、DPO、RLVR 与 multi-task evaluation 公开成可复现 recipe，并明确报告哪些方法“不稳定地提升”。[3] 对 Flywheel 来说，这一点很重要：闭环不是“任何反馈都训练”，而是要积累 negative knowledge——哪些 recipe、数据或超参在什么条件下没有收益。

15. Evaluator Flywheel：Judge / Verifier 自身也要持续改进

如果 evaluator 永远固定，policy 迟早会遇到两类问题：一是模型能力超过旧 grader；二是训练逐渐 exploit grader 的盲点。因此飞轮必须有 evaluator lane。

Evaluator Flywheel	动作
Disagreement mining	采样 human vs LLM judge、judge vs verifier、multi-judge 不一致案例
Calibration set	维护高质量专家标注的私有校准集
Hard-negative generation	专门生成“看起来好但实际错”的 response
Judge versioning	每个 reward/eval 结果绑定 grader_version
Shadow evaluation	新 judge 先影子运行，比较旧 judge / human
Evaluator training	必要时用 synthetic + human feedback 更新 judge/RM/PRM

Self-Taught Evaluators 展示了 evaluator 自我改进的可能性：从无标签 instruction 出发，迭代生成 contrasting outputs，并训练 judge 产生 reasoning + judgment；论文报告在 RewardBench 上把 Llama3-70B-Instruct evaluator 从 75.4 提升到 88.3（majority vote 88.7），且不使用人工 preference labels。[7] 这并不意味着可以取消 human calibration，而是说明 evaluator 也可以进入 data flywheel。

16. RL 数据 Flywheel：从均匀采样到 Learning Frontier 控制

在 RLVR/GRPO 中，数据飞轮发生在“每一步 rollout”层面。对每个 prompt 均匀分配 N 个 rollout 很浪费：已掌握题全对，超难题全错，group-relative advantage 都会退化。

状态	Group / Reward 现象	控制动作
全对	低方差、近零学习信号	降采样/转 regression
全错	低方差、没有正轨迹	降低难度、teacher bootstrapping、curriculum

状态	Group / Reward 现象	控制动作
混合成功/失败	高信息量	加大 rollout / gradient budget
reward 波动但 grader 不稳	可能是假 signal	grader audit / human escalation
训练后 pass rate 快速上升	已经跨过 frontier	动态调低权重，寻找下一批难题

DAPO 的 Dynamic Sampling 明确过滤 group 全对/全错的 prompt，以维持有效训练信号。[8] 2026 LZE 则进一步根据 difficulty、outcome uncertainty 与 pass-rate momentum 做在线 prompt budget 分配。[9] 两者共同说明：RL 数据选择不是预处理，而是训练过程中的实时控制策略。

17. Agent Flywheel：训练单位从 Sample 变成 Trajectory + Environment

Agent 信号	需要记录什么	如何转训练
Tool selection error	候选工具、调用时机、schema、结果	Tool SFT / pair / process reward
Planning failure	计划、subgoal、后续状态	trajectory preference / agent RL
Recovery failure	错误后 observation 与恢复动作	failure-repair trajectory SFT
Environment exploit	最终 state 与未预期路径	修 verifier/harness + adversarial data
Cost blow-up	turn/tool/token/latency	cost-aware reward / policy constraint
Long-horizon drift	first divergence + propagated errors	critical-step labeling / process supervision

Anthropic 的 Agent Eval 框架强调 transcript 与 outcome 必须同时记录：模型说“已预订”并不等于数据库里真的存在预订。[2] 对飞轮来说，Agent 的“标签”往往来自环境 end state、unit tests、state checks，而不是 final answer 文本本身。

18. Production Data Flywheel: Online / Offline 两条车道

Offline Lane	Online / Production Lane
稳定 benchmark / regression	真实用户输入、输出、outcome
人工构造 challenge set	用户反馈、业务结果、监警告警
可复现、低噪声	新鲜但高噪声、无天然 ground truth
适合研究/ablation	适合发现 unknown unknowns
更新慢	持续流入
主要风险: benchmark overfit	主要风险: 隐私、偏差、label leakage、反馈选择偏差

推荐做法不是二选一，而是“双车道合流”：production lane 负责发现新 gap，offline lane 负责验证、复现、归因与形成稳定 regression。OpenAI 的 eval/data flywheel 建议也强调在真实输出上持续测量，并把日志中模糊或高成本案例路由给专家，再回写 eval/error analysis。[1]

19. 数据飞轮的隐私、治理与污染边界

风险	典型问题	治理机制
Privacy	生产日志含个人/敏感信息	最小化采集、脱敏、访问控制、保留期限
Consent / Policy	用户数据是否允许训练用途	明确用途与数据处理政策
Eval Leakage	失败样本同时进入 train 与 private eval	dataset lineage + hash/semantic contamination check
Feedback Bias	thumbs-down 过度代表极端用户	stratified sampling + product outcome 补充
Synthetic Monoculture	模型生成数据越来越同质	generator diversity + real anchor + provenance
Judge Leakage	训练模型猜到 rubric / hidden tests	grader isolation、hidden eval、adversarial audit

Flywheel 越自动化，治理越重要：错误一次被写回数据，就可能在下一轮变成“模型先验”，再影响后续采样与 evaluator，形成自强化偏差。生产系统需要把 provenance 和 data-use policy 设计为一等字段，而不是训练前临时清洗。

20. 经济学：最应该优化的是 Learning Gain / Total Cost

$$\text{Flywheel ROI} \approx \text{Verified Product/Capability Gain} / (\text{Collect} + \text{Label} + \text{Generate} + \text{Verify} + \text{Train} + \text{Eval} + \text{Regression Cost})$$

成本项	容易被忽略的部分
Collection	生产采样、存储、隐私处理、专家时间
Generation	best-of-N、长 CoT、Agent environment interaction
Verification	unit tests、judge calls、human escalation
Training	GPU/FLOPs、checkpoint、失败实验

成本项	容易被忽略的部分
Evaluation	多 trial、复杂环境、统计置信度
Regression	全套历史能力反复重跑
Opportunity Cost	训练一个低价值 gap 占用的模型与团队周期

因此“自动闭环”不等于“所有失败自动训练”。最成熟的系统应该有 abstain / defer：当 confidence、trainability 或 verifier reliability 低时，把问题留在 gap queue，先改 evaluator、收更多证据，或者直接修 product/harness。

21. Flywheel KPI：不要用“数据量”作为主指标

KPI	含义
Verified Failure Precision	进入训练队列的 failure 中，真实模型问题的比例
Gap-to-Data Lead Time	一个新 failure family 到可训练数据的时间
Accepted Signal Rate	生成/采集数据中通过 verifier/filter 的比例
Marginal Gain / 1k examples	单位数据的增益
Marginal Gain / GPU-hour	单位训练资源的增益
Regression Escape Rate	发布后才发现旧能力回退的比例
Freshness Lag	训练数据与当前 production policy 的版本/时间差
Coverage Delta	新一轮数据带来的 capability/failure coverage 增量
Fix Generalization	对 validation/OOD failure family 的修复率，而非只修原样本
Rollback Rate	因隐藏回归而撤销 checkpoint 的频率

22. 18 类 Data Flywheel 常见失败模式

# / 模式	表现	后果
1. Badcase Dump	所有失败直接回训	把噪声/规格错误写进模型
2. Benchmark Tunnel Vision	只追公开分数	真实产品不提升
3. No Root Cause	不区分 tool/eval/model	错误路由训练阶段
4. Stale Preference	长期用旧 pair	对当前 policy 梯度弱
5. No Replay Anchor	只训新问题	灾难性遗忘
6. Eval Leakage	train/eval 混用	分数虚高
7. Reward Monoculture	单 judge/单 reward	模型学会 exploit proxy
8. Synthetic Monoculture	单 generator 递归造数	长尾消失
9. Overfit to One Failure	只复制原 badcase	修样本不修 family
10. No Benign Counterpart	只训拒绝/防御	过拒绝 / false positive
11. Fixed Task Mix	模型变强后比例不变	预算浪费在已掌握区
12. Uniform RL Sampling	所有 prompt 相同预算	大量 zero-gradient experience
13. No Experiment Memory	每轮忘记历史结果	重复失败实验
14. Grader Drift	grader 更新无版本	无法比较历史结果
15. Production Bias	只采 thumbs-down	样本分布失真
16. No Cost Accounting	只看质量不算 verifier/rollout 成本	飞轮不可规模化
17. Release Without Gates	局部指标好就发布	回归逃逸
18. Automation Without Abstain	低置信问题也自动训练	错误自强化

23. L0-L4 Data Flywheel 成熟度模型

等级	系统形态	关键能力	典型瓶颈
L0 Pile	人工收 badcase + 偶尔微调	有问题意识	无 lineage、无闭环
L1 Eval-driven	有 eval/error taxonomy	失败可量化	数据/训练路由仍人工
L2 Routed Flywheel	Failure validation + stage router + regression	能稳定把 gap 编译成训练信号	freshness/成本控制不足
L3 Adaptive Controller	动态 task mix、near-policy data、自动优先级	根据当前模型状态分配数据/ compute	需要强 evaluator 与 infra
L4 Self-improving Platform	自动 failure search、adaptive generation/training/eval、human escalation	持续寻找 frontier，同时保护 retention	治理、reward hacking、系统复杂性

24. 推荐的生产级系统架构

可以把 Data Flywheel 实现成六个逻辑服务，而不是一个巨大的“训练脚本”：

服务	职责	关键状态
Signal Collector	统一采集 eval/prod/red-team/user events	event_id, model/harness version
Failure Validator	复现、grader/harness/spec audit	confidence, root-cause
Gap Queue Controller	cluster、priority、budget	failure family, priority, SLA
Data Compiler	生成/检索/验证 SFT/pair/trajectory/reward data	dataset lineage, verifier result
Training Orchestrator	SFT/DPO/RLVR/distill、实验与 checkpoint	recipe, seed, data version
Eval & Release Gate	capability/regression/product/OOD	release decision, rollback reason

架构原则： 所有服务通过“版本化事件与数据契约”连接，而不是靠文件名和口头约定。模型版本、grader 版本、harness 版本、dataset 版本、训练 recipe 版本必须可追溯。

25. 可直接复用的 Data Flywheel Contract（示例）

```
flywheel_run:
  objective_version: obj_v12
  source_model: model_2026_08_01
  signal_window: 2026-08-01..2026-08-07

collectors:
  - capability_eval
  - regression_eval
  - production_shadow
  - user_feedback

validation:
  min_reproducibility: 0.80
  require_grader_audit: true
  exclude_harness_failures: true

priority:
  factors: [impact, frequency, confidence, trainability, novelty, verifier_reliability]
  budget_unit: gpu_hour

routing:
  behavior_gap: sft
  preference_gap: dpo
  verifiable_gap: rlvr
  teacher_solved: distillation
  system_bug: product_fix

retention:
  regression_gate: required
  replay_mix: 0.30
  capability_drop_limit: 0.5pp

release:
  require_ood_gain: true
  require_product_shadow: true
  rollback_on_severity_1_regression: true
```

26. 90 天落地路线：从“人工闭环”到“可自动化闭环”

阶段	目标	最小交付
Day 0-30	建立可信传感器	50-200 个 capability/regression tasks; 统一 failure event; 人工 transcript review; 基础 taxonomy
Day 31-60	建立 failure-to-data 路由	Gap Queue; SFT/DPO/RLVR router; dataset lineage; 3-5 个可复现专项修复实验
Day 61-90	建立动态控制与 release gate	near-policy generation; replay bank; 自动 regression; priority/budget controller; production shadow
之后	逐步自动化	adversarial failure search、adaptive task mix、judge/evaluator flywheel、human escalation

不要一开始追求“全自动自我改进”。先确保每条链路可审计、可复现、能回滚；只有当 failure precision、grader calibration 与 regression gate 足够稳定后，才值得逐步把生成、选择与训练决策自动化。

27. 最终 Know-Why / Know-How 总表

问题	Know-Why	Know-How
为什么要飞轮？	静态数据不能长期覆盖不断变化的模型与产品分布	让数据分布成为当前 policy 状态的函数
为什么先验证失败？	低分可能来自 grader/harness/spec	复现 + transcript + grader audit
为什么要 Gap Queue？	训练预算有限	多因子 priority + cost-aware budget
为什么要 stage router？	不同 failure 需要不同监督信号	SFT / DPO / RLVR / Distill / Product Fix
为什么要 near-policy data？	旧数据与当前 policy coverage 不匹配	迭代 candidate generation / online sampling
为什么要 replay/regression？	持续训练必然有遗忘风险	replay anchor + regression bank + release gate
为什么要 evaluator flywheel？	模型会超过/利用固定 grader	disagreement mining + calibration + versioning
为什么要 cost-aware？	高质量 signal 可能比训练本身更贵	优化 verified gain / total cost
为什么要 human escalation？	自动 judge/生成永远存在未知盲点	对高影响/高不确定案例保留专家通道

最终心智模型： Data Flywheel 的目标不是让训练“永不停止”，而是建立一套能持续判断“下一单位训练预算应该花在哪里”的反馈控制系统。成熟系统会自动停止对低价值问题投入，也会在信号不可信时选择不训练。

28. 核心参考资料与阅读路线

- [1] OpenAI. How evals drive the next chapter in AI for businesses. 2025-11-19. 关键：Specify → Measure → Improve；明确提出 data flywheel。
- [2] Anthropic. Demystifying evals for AI agents. 2026-01-09. 关键：capability vs regression、transcript/outcome、production monitoring、eval lifecycle。
- [3] Lambert et al. Tulu 3: Pushing Frontiers in Open Language Model Post-Training. arXiv:2411.15124. 关键：SFT→DPO→RLVR、开放 data/eval recipe。
- [4] Grattafiori et al. The Llama 3 Herd of Models. arXiv:2407.21783. 关键：六轮 post-training、最新 policy rejection sampling、数据复杂度随模型能力升级。
- [5] Zhang, Sleight, Benton. A3: An Automated Alignment Agent for Safety Finetuning. Anthropic Fellows, 2026-03-11. 关键：failure hypothesis、adaptive data weighting、OOD split、retention。
- [6] OpenAI. GPT-Red: Unlocking Self-Improvement for Robustness. 2026-07-15. 关键：self-play red-teaming→adversarial training→更强 defender→更强 attacker。
- [7] Wang et al. Self-Taught Evaluators. arXiv:2408.02666. 关键：evaluator 自我改进与 synthetic preference feedback loop。
- [8] Yu et al. DAPO: An Open-Source LLM Reinforcement Learning System at Scale. arXiv:2503.14476. 关键：Dynamic Sampling 与 long-CoT RL 稳定化。
- [9] Cui, Yang, Zhu. Learning-Zone Energy: Online Data Selection for Efficient RL Post-Training. arXiv:2605.17003. 关键：在线 learning frontier 与 compute-aware prompt selection。
- [10] Chen et al. On-Policy Replay for Continual Supervised Fine-Tuning. arXiv:2605.29495. 关键：持续 SFT 的 on-policy replay 与 遗忘控制。
- [11] Zeng et al. ARIES: Stimulating Self-Refinement of LLMs by Iterative Preference Optimization. arXiv:2502.05605. 关键：迭代 preference + self-refinement data collection。
- [12] Liu et al. Iterative Length-Regularized Direct Preference Optimization. arXiv:2406.11817. 关键：iterative online preference 与 verbosity side effect。
- [13] Djuhera et al. Fixing It in Post: A Comparative Study of LLM Post-Training Data Quality and Model Performance. arXiv:2506.06522. 关键：更少但更好/更匹配的数据混合。

推荐阅读顺序：先读 [1][2] 建立“评测驱动闭环”心智模型；再读 [4][3] 看真实 post-training recipe；随后读 [5][6] 理解自动 failure→data→train；最后读 [8][9][10][11] 进入动态采样、持续学习与 self-improvement。

— End —